Problem A. Counting Permutations

For every valid permutation, the sequence of pairs

$$(a_{p_i} \bmod m_1,\ a_{p_i} \bmod m_2)$$

is forced. It is exactly the sequence obtained by sorting all elements increasingly by the first key $a_i \bmod m_1$ and then by the second key $a_i \bmod m_2$.

If this pair sequence is invalid, the answer is 0. Otherwise, elements with the same pair may be permuted freely. Hence the number of valid permutations is the product of $c!$ over all multiplicities c of equal pairs.

The time complexity is $O(n \log n)$.

Problem B. Bookshelf Tracking

If there are only operations of type E, maintain the position of every book; then each exchange can be processed in $O(1)$ time.

When type R operations exist, observe what an R operation really does: it sorts the books in the first half by decreasing label and the books in the second half by increasing label. Therefore, before the last R operation, we only need to maintain whether each book is in the first half or the second half. After performing the final sort, process the last consecutive block of E operations normally.

The time complexity is $O(n + q)$.

Problem C. Painting Fences

An optimal solution starts by expanding from a maximal all-black rectangle. There are $O(nm)$ such rectangles; they can be found, for example, by enumerating the bottom side and using a Cartesian tree.

Now consider the number of operations needed to expand one rectangle to the whole board. The two dimensions are independent, so it is enough to solve the one-dimensional problem: expand an interval $[l, r]$ to $[0, n]$.

If $l = 0$ or $r = n$, the optimal strategy is clearly to double the covered length in every step.

For the general case, enumerate which boundary, $l = 0$ or $r = n$, is reached first. Suppose $l = 0$ is reached first. Let

$$d = \left\lceil \frac{l}{r - l} \right\rceil.$$

At least $\lceil \log_2(d + 1) \rceil$ steps are necessary. However, always reflecting to the left may waste length. The optimal rule is the following: in the i-th round, if the i-th bit of d from low to high is 1, reflect to the left; otherwise, reflect to the right.

Thus the minimum number of expansion steps from a fixed rectangle can be computed in $O(\log nm)$ time, or even $O(1)$ with preprocessing and casework.

The total time complexity is $O(nm \log nm)$, or $O(nm)$ with the constant-time expansion formula.

Problem D. Function with Many Maximums

First note that, if a_i is sorted in decreasing order, then

$$f(a_i) = \sum_{j=1}^{i} a_j + i a_i.$$

If the integrality condition on a_i is ignored, choosing $a_i = \frac{1}{i(i+1)}$ makes all values $f(a_i)$ equal.

Take a large integer M, roughly of order Cn^4, and initialize

$$a_i = \left\lfloor \frac{M}{i(i+1)} \right\rfloor.$$

Then adjust this sequence. We want

$$f(a_n),\ f(a_{n-2}),\ f(a_{n-4}), \ldots, f(a_{n-2(b-1)})$$

to be the maximum values. For $i < n - 2(b-1)$, it is enough to set $a_i = a_{i+1} + 1$. At this moment, the largest value a_1 is only $O(Cn^2)$, so it satisfies the required bound.

For the suffix, first make the sequence of function values increasing. Whenever a drop appears, increase the corresponding a_i by $O(n)$. This works because adjacent function-value differences are originally $O(n)$, so the accumulated correction is $O(n^2)$, and increasing a_i by 1 increases $f(a_i)$ by $i + 1$.

After this correction, adjacent function-value differences are still $O(n)$, while adjacent a_i differ by $\Theta(Cn)$. Therefore decreasing a_{2i+1} by $f(a_{2i+2}) - f(a_{2i})$ makes $f(a_{2i+2}) = f(a_{2i})$ without breaking the construction.

Problem E. Building a Fence

We split into cases. Assume without loss of generality that $w \le h$.

- If $s \le w \le h$, greedily place sticks starting from a corner. No stick of length s has to be split into three pieces, so the lower bound
$$\left\lceil \frac{2(w+h)}{s} \right\rceil$$
is attainable.
- If $w \le h < s$, then the answer is one of $2, 3, 4$.
  - The answer can be 2 only when both sticks are split into two parts, i.e. $w + h = s$.
  - For the answer to be 3, there must be k sticks that exactly cover $k + 1$ sides.
    - $k = 1$: $2w = s$ or $2h = s$.
    - $k = 2$: $2w + h = 2s$ or $2h + w = 2s$.
    - $k = 3$: $2(w + h) = 3s$.
  - Otherwise the answer is 4.
- If $w < s \le h$, then:
  - If $2w + h \ge 2s$, use two sticks to cut out the two sides of length w, put the remaining parts on the same long side, and then greedily continue from that side. This achieves the same lower bound $\lceil 2(w + h)/s \rceil$.
  - Otherwise the answer is 3 or 4. The same analysis as above applies, but due to the size relation only the case $k = 3$ is possible, namely $2(w + h) = 3s$.

Problem F. Teleports

After any operation, the possible positions of the treasure form an interval. Let $f(l, r)$ be the minimum cost to find the treasure when it is known to be in boxes $l, l + 1, \ldots, r$.

Enumerate the chosen teleporter x. The nominal new possible range is $[2x - r, 2x - l]$.

- If $1 \le 2x - r \le 2x - l \le n$, then
$$f(l, r) \le f(2x - r, 2x - l).$$
- If $2x - r < 1$, then the treasure in $[2x, r]$ cannot be teleported, and
$$f(l, r) \le \max\{f(1, 2x - l),\ f(2x, r)\}.$$
- If $2x - l > n$, then the treasure in $[l, 2x - n - 1]$ cannot be teleported, and
$$f(l, r) \le \max\{f(2x - r, n),\ f(l, 2x - n - 1)\}.$$

The transition graph is layered by interval length $r - l$. Inside one layer, only the first type of transition appears. Therefore, process interval lengths increasingly: first apply all transitions of the second and third

types, then run Dijkstra for the first type. The graph inside a layer is dense, so the $O(n^2)$ implementation of Dijkstra is sufficient, although a heap implementation also works well.

The time complexity is $O(n^3)$.

Problem G. Exponent Calculator

Only addition, subtraction, and multiplication are available, so Newton iteration is not considered. Instead, approximate

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

This converges slowly when $|x|$ is large. Thus first divide x by 2^B, expand the Taylor series up to the x^T term, and finally square the result B times.

Experiments show that $T = 7$ and $B = 10$ are a good choice: even for $x = 20$, the relative error can be pushed below 10^{-12}.

Problem H. Ancient Country

Consider the vertices belonging to all cities. It is impossible to have indices $i < j < k < l$ such that P_i and P_k are vertices of the same city, while P_j and P_l are vertices of another same city; otherwise the two polygonal edges would intersect or leave the country.

Hence the city-vertex structure is tree-like. If the city containing the smallest vertex has vertices

$$P_{i_1}, P_{i_2}, \ldots, P_{i_k},$$

then all remaining city vertices must lie in the intervals

$$[i_1 + 1, i_2 - 1],\ [i_2 + 1, i_3 - 1], \ldots, [i_k + 1, n].$$

This gives an interval DP. Let $f(l, r)$ be the maximum protection level that can be constructed using only vertices with indices from l to r.

- If P_l does not belong to any city, update $f(l, r) \leftarrow f(l + 1, r)$.
- Otherwise, choose a convex hull containing P_l. Add its weight and the DP values of the intervals between consecutive hull vertices. This choice can also be optimized by DP.

For the second case, first enumerate the first edge P_iP_s of the hull. Let $g(j, k)$ be the maximum weight when the last edge of the current hull is P_jP_k. The transition enumerates the next edge P_kP_e and moves to $g(k, e)$. Finally, check whether connecting back to l is valid and update the answer. The first edge is enumerated because the angle $P_kP_iP_s$ must be checked. Since the indices are increasing, it is enough to check angles of adjacent triples; self-intersections cannot occur.

In implementation, enumerate l from large to small. For fixed l, s, the DP is $O(n^3)$. The transition can be optimized by angular order: for fixed k, e, the possible previous vertices form a prefix. Because indices increase, edges from a point are already sorted by angle when sorted by index, so a two-pointer scan suffices.

The total time complexity is $O(n^4)$.

Problem I. Marks Sum

If the input length is less than 2000, directly return $d = info = 0$.

If the length is at least 2000, we can always find a prefix such that the remaining length is less than 2000, and the sum of the prefix is either a multiple of 2000 or one more than a multiple of 2000. Let

$$info = 2x + y \qquad (0 \le y \le 1)$$

represent that the prefix sum is $2000x + y$. Since the string length is at most 10^6, we have $x < 1000$, so this information fits.

On the second call, use $info$ to recover the prefix sum and add the sum of the current input string.

Problem J. Prefix Divisible by Suffix

Enumerate the shortest suffix that satisfies the condition. If the suffix has leading zeroes, removing them still preserves the condition. Therefore the shortest valid suffix has length at most 7; if it were longer, the prefix would be smaller than the suffix.

Because the enumerated suffix is required to be the shortest one, use inclusion-exclusion over all shorter suffixes. The numbers satisfying several suffix conditions have the form $km + a$, and the parameters can be obtained by the extended Euclidean algorithm.

The time complexity is $O(10^7 \cdot 2^7 \cdot \log n)$. In practice, when the suffix is long, many subsets already force the number to exceed n, so they can be pruned and the actual number of enumerations is much smaller.

Problem K. Train Depot

First convert every train into a path from s_i to t_i, where t_i is an ancestor of s_i, with value c_i. The task is to choose vertex-disjoint paths with maximum total value.

Let $f(x)$ be the maximum value obtainable inside the subtree of x. If x is not the top endpoint of a chosen path, then

$$f(x) \leftarrow \sum_{y \in ch(x)} f(y).$$

Suppose we choose the path $x \to y$. Then every sibling of each vertex on the path from y to x (excluding x), and every child z of y, contributes $f(z)$. Conversely, each vertex x contributes to paths whose bottom endpoint lies either at the parent of x or inside a sibling subtree of x. In DFS order, these are exactly the intervals

$$[in_{parent_x}, in_x) \quad \text{and} \quad [out_x, out_{parent_x}).$$

Maintain these contributions using a segment tree or a Fenwick tree.

The time complexity is $O((n + \sum k_i) \log n)$.

Problem L. Array Spread

Binary-search the answer x. If there exists a sequence such that every specified interval sum lies in $[1, x]$, then the answer is at most x.

Let s_i be the prefix sum of the first i numbers. The constraints are

$$s_i \geq s_{i-1}, \qquad 1 \leq s_{r_i} - s_{l_i - 1} \leq x,$$

which are difference constraints. After coordinate compression, the number of vertices and edges is $O(m)$, so detecting a negative cycle takes $O(m^2)$ time.

This also shows that the answer is a rational number whose numerator and denominator are both $O(m)$: the denominator is bounded by the length of a simple cycle, and the numerator is bounded by the total non-x edge weights on such a cycle. Thus $O(\log m)$ binary-search steps are enough to recover the exact rational value after the real-valued search. Alternatively, one can binary-search directly over the $O(m^2)$ candidate fractions.

This gives $O(m^2 \log m)$ time.

The binary search can also be removed. Call edges with weight x special, and all other edges ordinary. Let $f(i, v)$ be the minimum total ordinary-edge weight of a path that starts anywhere, uses exactly i special

edges, and ends at v. If n is the number of vertices, then the answer is

$$\max_{v} \min_{i=0}^{n-1} \frac{f(i,v) - f(n,v)}{n-i}.$$

Problem M. Uniting Amoebas

Repeatedly merge a maximum element with one of its neighbors. This achieves the value

$$\sum_{i=1}^{n} a_i - \max_{1 \le i \le n} a_i.$$

This is optimal: paying a merge cost of v can increase the current maximum by at most v. Therefore the total cost must be at least the total sum minus the final maximum, and the greedy process reaches this lower bound.